

Adaptive CPU Auto-Scaling Using Hybrid Neural Networks and Continual Learning in Kubernetes Orchestration

Research Proposal

*Submitted in partial fulfilment of the requirement for the degree of Bachelor of
Science Honours in Information Technology*

By:
U.K Nalawansa
2020/ICT/07

**Department of Physical Science
Faculty of Applied Science
University of Vavuniya
Sri Lanka**

December, 2025

1 Declaration

I, **Udara Kavishka Nalawansa (2020/ICT/07)**, do hereby declare that the work reported in this research proposal was carried out independently by me under the supervision of **Ms. R. Yasotha, Senior Lecturer, Department of Physical Science, Faculty of Applied Science, University of Vavuniya** and **Dr. S. Kirushanth, Senior Lecturer, Department of Physical Science, Faculty of Applied Science, University of Vavuniya**. This proposal describes the results of my own research, except where due reference has been made in the text. No part of this research project has been submitted earlier or concurrently for the same or any other degree.

Date: _____

Signature: _____

Certified by:

1. Supervisor: Ms. R. Yasotha

Date: _____

Signature: _____

2. Supervisor: Dr. S. Kirushanth

Date: _____

Signature: _____

Approved by IT Coordinator/ IT4216

Name: Mr. T. Jeyamugan

Date: _____

Signature: _____

Approved by IT Coordinator/ IT Degree Program

Name: Mr. T. Jeyamugan

Date: _____

Signature: _____

Approved by Head/ Department of Physical Science

Name: Mr. N. Edwin Linosh

Date: _____

Signature: _____

2 Introduction

2.1 Background

Kubernetes is the dominant container orchestration platform for managing cloud-native applications at scale. It automates deployment, scaling, and operation of containerized workloads while handling resource allocation, load balancing, and self-healing. This has made Kubernetes fundamental to CI/CD(Continuous Integration and Continuous Delivery/Deployment) pipelines, where applications must reliably scale to meet changing demand while maintaining cost efficiency[1].

Managing CPU (Central Processing Unit) resources in Kubernetes has some significant challenges. Traditional fixed resource allocation leads to over-provisioning and wasted spending, while under-provisioning causes performance degradation and SLA (Service Level Agreement) violations. Standard Kubernetes auto-scaling mechanisms like HPA(Horizontal Pod Autoscaler) operate reactively, adding pods only after CPU utilization exceeds its thresholds. This causes performance issues before scaling takes effect and fails to take any actions upcoming resource spikes.

Adaptive auto-scaling uses predictive approaches to forecast future resource needs and pre-allocate capacity before problems arise. LSTM (Long Short-Term Memory) networks have shown strong performance in time-series CPU forecasting. However, production workloads are non-stationary, where patterns can change due to seasonal effects, application updates, and user behavior changes. Static models lose accuracy as workloads change from their training distribution, requiring frequent retraining that is operationally expensive and introduces periods of degraded performance[2], [3].

Hybrid neural networks and continual learning offer a solution to this stability-plasticity dilemma. Combining LSTM for temporal patterns with MLPs (Multi Layer Perceptron) for static features captures both sequential dependencies and contextual information. Continual learning methods like Elastic Weight Consolidation (EWC) and Experience Replay (ER) enable adaptation to new patterns while preserving prior knowledge, preventing catastrophic forgetting. EWC protects important parameters from updates, while ER revisits past workload samples during new training.

This research integrates hybrid neural networks with continual learning for adaptive CPU management in Kubernetes CI/CD pipelines. While continual learning has succeeded in robotics and computer vision, its application to DevOps(Development and Operations) resource management remains unexplored. This approach enables continuous adaptation to evolving workloads without forgetting historical behaviors, improving scaling accuracy, reducing over-provisioning, and enhancing system reliability.

2.2 Problem Definition

Despite progress in machine learning-based auto-scaling, current systems face major limitations in real DevOps environments where static models trained on historical data degrade as workloads evolve through application updates, user behavior shifts, and infrastructure changes, requiring costly periodic retraining that causes catastrophic forgetting where it causes the models to lose their ability to handle baseline or recurring patterns after learning from recent data. Most solutions has a single-architecture designs (LSTM or MLP alone) rather than hybrid approaches, fall back to reactive threshold-based scaling that responds only after performance degradation occurs, The core challenge is to develop a CPU forecasting and auto-scaling system for Kubernetes that can adapt continuously to evolving workloads without forgetting established patterns, while lowering cost and improving reliability in DevOps environments.

This problem decomposes into:

- The limits of static models and reactive scaling.
- The plasticity–stability trade-off.
- DevOps-specific workload challenges.

A suitable solution must:

- Forecast temporal CPU demand and static job context.
- Adapt online to new workload patterns while retaining past knowledge.
- Integrate smoothly with Kubernetes-based DevOps tools.

2.3 Research Objectives

The main goal is to develop and evaluate a continual learning-based hybrid neural framework for adaptive CPU auto-scaling in Kubernetes.

Design a predictive auto-scaling system that:

- Forecasts CPU demand using a hybrid neural model.
- Uses EWC and ER to mitigate catastrophic forgetting under changing workloads.
- Improves Kubernetes scaling decisions beyond threshold-based methods.

Specific Objectives

1. Data and Feature Engineering

- Build time-series CPU usage and static features from Alibaba Traces.
- Engineer temporal and static feature sets for dual-branch modeling.

2. Hybrid Neural Architecture

- Develop an LSTM branch for temporal sequences and MLP branch for static features.
- Fuse branches to produce CPU forecasts.

3. Continual Learning Integration

- Apply EWC to preserve important parameters and Experience Replay with memory buffer.
- Combine both techniques for balanced adaptability and retention.

4. Training and Evaluation

- Train on sequential workload chunks simulating real-time learning.
- Compare against naive models, periodically retrained models, and Kubernetes HPA.

5. Kubernetes Integration

- Define how predictions drive Kubernetes replica management with stable scaling policies.

6. Performance Evaluation

- Evaluate CPU provisioning accuracy, SLA compliance, CI/CD execution time, and cost benefits.

2.4 Motivation

The motivation for this research arises from both practical needs and academic opportunities, as cloud environments face high economic costs due to frequent overprovisioning, operational instability caused by reactive scaling delays, reduced developer efficiency from slow CI/CD pipelines, and environmental concerns linked to unnecessary energy consumption. At the same time, the academic space presents several unexplored areas, including the integration of established methods such as LSTMs, continual learning, and Kubernetes autoscaling within DevOps workflows, and the extension of continual learning techniques into time series resource management where forgetting affects real operational performance. The study also evaluates a hybrid LSTM and MLP architecture to determine whether it enhances forecasting accuracy for resource management tasks, contributes reproducible datasets and evaluation protocols to address the lack of standardized benchmarks for continual learning based autoscaling, aligns technical improvements with measurable business outcomes such as cost reduction and SLA compliance, and introduces a methodological framework that includes structured data mining, knowledge retention through EWC and ER, and a multiobjective evaluation approach that considers accuracy, responsiveness, efficiency, and business impact.

3 Related Work

3.1 Machine Learning-Based Auto-Scaling in Cloud and Container Environments

Machine learning approaches to auto-scaling have evolved greatly over the past decade, increasing from simple regression models to more advanced deep learning architectures.

3.1.1 Early Statistical Approaches

In the beginning predictive auto-scaling research used time-series forecasting methods such as ARIMA (AutoRegressive Integrated Moving Average), Holt-Winters exponential smoothing, and linear regression. Wang (2021) compared Holt-Winters and LSTM for vertical CPU autoscaling in Kubernetes, finding that while Holt-Winters provided reasonable accuracy for periodic workloads, it struggled with non-stationary patterns and required manual parameter tuning. These statistical methods demonstrated the value of prediction-based scaling but doesn't have the capacity to model complex, non-linear workload dynamics characteristic of modern cloud environments[4], [5].

3.1.2 Machine Learning Era

The introduction of supervised machine learning techniques marked significant progress. Noor et al. (2019) proposed a hybrid auto-scaling system combining Random Forest for workload classification with Multiple Linear Regression for resource prediction. Their approach achieved 15-20% improvement over threshold-based methods but required extensive feature engineering and domain expertise [6].

Toka et al. (2021) developed a Kubernetes-specific scaling engine using neural networks and multiple linear regression that made application-agnostic scaling decisions handling the actual variability of incoming requests. Their work validated that ML-based approaches could generalize across diverse application types without manual tuning, addressing a key limitation of rule-based systems[7].

3.1.3 Deep Learning Advancement

BHyPreC, introduced in 2021, presented a novel hybrid recurrent neural network stacking Bi-LSTM on top of LSTM and GRU layers for CPU workload prediction in cloud VMs. This multi-layer hybrid architecture enhanced non-linear data analysis capability, demonstrating that combining multiple RNN variants could capture complex temporal patterns better than single architectures[8].

Recent work by Singh et al. (2025) integrated LSTM-based autoscaling with Integer Linear Programming (ILP) optimization for joint autoscaling and scheduling in KubeEdge environ-

ments. Tested on an 11-node testbed, their approach used CPU, memory, and RTT metrics for prediction while optimizing container placement through ILP, addressing the coupled problem of when and where to scale resources[9].

3.1.4 Production Deployment Validation

Alibaba’s AHPA (Adaptive Horizontal Pod Autoscaling) system, deployed on Alibaba Cloud Container Service for Kubernetes in 2023, represents a significant industry validation. Using robust decomposition forecasting to solve the elastic lag problem, AHPA achieved 10% increase in CPU usage and over 20% reduction in resource cost compared to previous algorithms across thousands of production workloads. This large-scale deployment demonstrated that ML-based autoscaling delivers measurable business value in real-world environments[10].

The MS-RA (Microservices Requirements-driven Autoscaling) solution proposed in 2024 achieved remarkable efficiency gains: 50% less CPU time, 87% less memory, and 90% fewer replicas compared to Kubernetes HPA while meeting SLO requirements. These dramatic improvements underscore the inefficiency of reactive threshold-based approaches and validate the superiority of intelligent predictive methods[3].

3.2 LSTM and Recurrent Neural Networks for Workload Prediction

Long Short-Term Memory networks have emerged as the dominant architecture for time-series workload forecasting due to their ability to capture long-range temporal dependencies.

3.2.1 Hybrid Recurrent Architectures

The BHyPreC model’s combination of Bi-LSTM, LSTM, and GRU demonstrated that heterogeneous recurrent architectures capture complementary temporal patterns. Their research showed that:

- GRU layers efficiently capture short-term fluctuations with fewer parameters
- LSTM layers model long-term dependencies and seasonal patterns
- Bi-LSTM layers refine predictions by incorporating bidirectional context

This architectural diversity enabled better handling of workload heterogeneity the coexistence of jobs with vastly different temporal characteristics within the same cluster[8].

3.3 Hybrid Neural Network Architectures

The combination of multiple neural network types has emerged as a promising direction for capturing diverse patterns in complex data.

3.3.1 Architectural Diversity Rationale

Ghimire et al. (2022) provided comprehensive analysis of CNN-LSTM-MLP hybrid fusion models for solar radiation forecasting, demonstrating that architectural diversity consistently outperforms single-method approaches. Their work showed:

- CNN layers excel at extracting spatial features and local patterns
- LSTM/GRU layers capture temporal dependencies and sequential dynamics
- MLP layers effectively process static features and perform non-linear transformations

Applying similar reasoning to workload prediction: LSTM captures temporal CPU usage patterns, while MLP processes static job attributes (priority, scheduling class, resource requests) that influence consumption but don't exhibit temporal variation[11].

3.4 Continual Learning and Catastrophic Forgetting

Continual learning addresses the challenge of adapting models to new data while preserving previously acquired knowledge, critical for production systems facing evolving workloads.

3.4.1 Experience Replay Mechanisms

Rolnick et al.'s (2018) CLEAR (Continual LEARNING with Replay) demonstrated that replay buffers can almost eliminate catastrophic forgetting. By maintaining a memory of representative past examples and mixing them with new data during training, Experience Replay ensures the model retains knowledge of previous patterns. The research showed that even constrained buffer sizes (storing only 1-5% of total data) provide substantial forgetting mitigation through intelligent sample selection[12].

3.4.2 Continual Learning for Time-Series

CEL (Continual Learning Model for Disease Outbreak Prediction) applied EWC with LSTM for domain incremental learning on time-series data. Results showed minimal 65% forgetting rate and 18% higher memory stability compared to standard retraining. This work demonstrated EWC's effectiveness for temporal prediction tasks, though in a different application domain (epidemiology rather than resource management)[13].

3.4.3 Auto-Scaling Specific Application

Hao et al.'s (2023) DMSHM (Density-based Memory Selection and Hint-based Network Learning Model) represents the only identified work explicitly addressing continual learning for predictive autoscaling [14]. Key contributions:

- Discovered sample overlap phenomenon: replay-based methods in prediction tasks exhibit unique challenges compared to classification tasks
- Introduced density-based sample selection using kernel density estimation for intelligent memory buffer construction
- Demonstrated that continual learning enables accurate predictions using only a small portion of historical logs

This work validates continual learning’s applicability to auto-scaling but focuses on general cloud environments rather than DevOps-specific contexts.

3.5 Kubernetes Auto-Scaling and Container Orchestration

Kubernetes has become the dominant container orchestration platform, but its default auto-scaling mechanisms exhibit significant limitations.

3.5.1 Kubernetes HPA Limitations

The default Horizontal Pod Autoscaler (HPA) uses reactive threshold-based scaling, monitoring CPU/memory utilization and scaling when thresholds are exceeded. Research has identified critical shortcomings:

- Reactive nature: Scales only after resource exhaustion, causing performance degradation
- Metric lag: Default 15-second metric collection intervals introduce delays
- Oscillation: Naive threshold rules cause rapid scale-up/scale-down cycles
- Lack of prediction: No anticipatory capability for known patterns (daily cycles, weekly trends [15])

3.5.2 Enhanced Kubernetes Scaling Solutions

Multiple research efforts have developed intelligent replacements for HPA:

AHPA (Alibaba): Deployed across Alibaba Cloud, uses robust decomposition forecasting to predict workload and proactively scale. Achieved 10% CPU usage increase and 20% cost reduction in production.[10]

MS-RA: Requirements-driven autoscaling achieving 50% less CPU, 87% less memory, 90% fewer replicas than HPA while meeting SLOs. Self-adaptive architecture adjusts to changing requirements without manual intervention[3].

4 Research Gaps

From past studies, we can see that:

- **Lack of Continual Learning for Kubernetes Scaling:** Almost no continual learning approaches like EWC or Experience Replay have been applied to Kubernetes auto-scaling, leaving the problem of adapting models without full retraining and without forgetting old knowledge unsolved.
- **Catastrophic Forgetting in DevOps is Unexplored:** While catastrophic forgetting is well-known in ML, it is rarely addressed in auto-scaling research, with most studies focusing on current data accuracy rather than retaining performance on previous workloads.
- **Hybrid Temporal–Static Modeling is Underused:** Most solutions use either time-series models on CPU data or feedforward models on static features, with few combining both temporal sequences and static job characteristics in hybrid architectures.
- **Limited Focus on DevOps/CI/CD Workloads:** Most auto-scaling research targets web services rather than DevOps workloads, which exhibit job-driven behavior, stage dependencies specific to CI/CD pipelines.

5 Materials and Methods

5.1 Materials/Tools

5.1.1 Dataset: Alibaba Cluster Traces

The Alibaba Cluster Traces (2025) provide detailed telemetry from a large-scale containerized cloud environment, containing instance-level and node-level information over a one-month period covering thousands of machines and tens of thousands of application instances. Each instance includes CPU, memory, disk, and GPU resource requests and limits, scheduling constraints, lifecycle timestamps, and application grouping. [16].

5.1.2 Simulation Environment

A Kubernetes cluster using Minikube. Alibaba trace jobs are replayed according to their arrival times. Prometheus and Grafana are used to collect metrics.

5.2 Procedures

1. **Data Preprocessing:** Parse and clean raw cluster trace data by removing incomplete records and filtering outliers.
2. **Hybrid Model Development:** Construct a hybrid neural network combining an LSTM branch for processing temporal CPU sequences with an MLP branch for handling static job features.
3. **Continual Learning Integration:** Implement a continual learning engine integrating Elastic Weight Consolidation (EWC) to protect important model parameters and Experience Replay (ER) to maintain a memory buffer of historical samples
4. **Auto-Scaling Controller Implementation:** Develop an auto-scaling controller that translates CPU predictions into rate-limited pod scaling decisions through the Kubernetes API
5. **Simulation Execution:** Initialize the Kubernetes cluster with baseline controllers and replay workload traces according to original temporal patterns. Deploy each auto-scaling method (HPA, static LSTM, static hybrid, periodic retraining, and proposed continual learning approach) and collect performance metrics including CPU usage, predictions, scaling actions, SLA violations, and cost.

5.3 Data Analysis

5.3.1 Evaluation Metrics

To evaluate model performance, prediction accuracy metrics including MAE (Mean Absolute Error), RMSE (Root Mean Square Error), and MAPE (Mean Absolute Percentage Error) are measured. Scaling efficiency is assessed through resource utilization and over-provisioning ratios. Operational quality is evaluated via average CPU utilization and response lag. SLA compliance is tracked through violation counts and duration. Cost metrics include total resource hours consumed. Continual learning effectiveness is measured through forgetting rates versus improvement on new patterns.

5.3.2 Baseline Methods for Comparison

The following baseline methods are used for comparison:

- Kubernetes Horizontal Pod Autoscaler (HPA)
- Static LSTM-only model
- Static hybrid model without continual learning
- Periodic retraining of the hybrid model

6 Expected Outcomes

The proposed framework is expected to achieve the following:

1. **Improved Prediction Accuracy:** Surpass naive and threshold-based methods by leveraging both temporal and static information.
2. **Reduced Catastrophic Forgetting:** EWC and ER mechanisms enhance retention of earlier workload patterns compared to naive fine-tuning.
3. **Stable and Cost-Efficient Auto-Scaling:**
 - Lower CPU over-provisioning without increasing under-provisioning risk.
 - More consistent SLA compliance under dynamically changing workloads.
4. **Enhanced DevOps Pipeline Performance (where tested):**
 - Shorter queue times for builds and tests.
 - More predictable pipeline execution times during peak hours.
5. **Methodological Contributions:**
 - A reusable hybrid continual learning architecture for resource forecasting.
 - An evaluation protocol

7 Gantt Chart

Work/Time (Months)	Nov-25	Dec-25	Jan-26	Feb-26	Mar-26	Apr-26	May-26	Jun-26
Supervisor Selection and Title Selection								
Proposal								
Literature Review								
Resource gathering								
Tools and Techniques								
Implementation								
Documentation Writing								
Publication / Conferences								
Presentation and Manuscript Submission								

References

- [1] Devesh Srivastava et al. “Auto-Scaling of Cloud Applications Using Machine Learning”. In: *2025 International Conference on Next Generation of Green Information and Emerging Technologies (GIET)*. Gunupur, India: IEEE, Aug. 2025, pp. 1–6. ISBN: 978-1-6654-5806-1. DOI: 10.1109/GIET65294.2025.11234879. URL: <https://ieeexplore.ieee.org/document/11234879/> (visited on 12/11/2025).
- [2] Mahmoud Imdoukh, Imtiaz Ahmad, and Mohammad Gh. Alfaiakawi. “Machine learning-based auto-scaling for containerized applications”. en. In: *Neural Comput & Applic* 32.13 (July 2020), pp. 9745–9760. ISSN: 0941-0643, 1433-3058. DOI: 10.1007/s00521-019-04507-z. URL: <http://link.springer.com/10.1007/s00521-019-04507-z> (visited on 12/11/2025).
- [3] João Paulo Karol Santos Nunes et al. *Self-adaptive, Requirements-driven Autoscaling of Microservices*. arXiv:2403.08798 [cs]. Feb. 2024. DOI: 10.48550/arXiv.2403.08798. URL: <http://arxiv.org/abs/2403.08798> (visited on 12/11/2025).
- [4] Abhishek Aich. *Elastic Weight Consolidation (EWC): Nuts and Bolts*. arXiv:2105.04093 [cs]. May 2021. DOI: 10.48550/arXiv.2105.04093. URL: <http://arxiv.org/abs/2105.04093> (visited on 12/11/2025).
- [5] Haibin Yuan and Shengchen Liao. “A Time Series-Based Approach to Elastic Kubernetes Scaling”. en. In: *Electronics* 13.2 (Jan. 2024), p. 285. ISSN: 2079-9292. DOI: 10.3390/electronics13020285. URL: <https://www.mdpi.com/2079-9292/13/2/285> (visited on 12/11/2025).
- [6] Adnan Umer, Adnan Noor Mian, and Omer Rana. “Predicting machine behavior from Google cluster workload traces”. en. In: *Concurrency and Computation* 35.5 (Feb. 2023), e7559. ISSN: 1532-0626, 1532-0634. DOI: 10.1002/cpe.7559. URL: <https://onlinelibrary.wiley.com/doi/10.1002/cpe.7559> (visited on 12/11/2025).
- [7] Laszlo Toka et al. “Machine Learning-Based Scaling Management for Kubernetes Edge Clusters”. In: *IEEE Trans. Netw. Serv. Manage.* 18.1 (Mar. 2021), pp. 958–972. ISSN: 1932-4537, 2373-7379. DOI: 10.1109/TNSM.2021.3052837. URL: <https://ieeexplore.ieee.org/document/9328525/> (visited on 12/11/2025).
- [8] Md. Ebtidaul Karim et al. “BHyPreC: A Novel Bi-LSTM Based Hybrid Recurrent Neural Network Model to Predict the CPU Workload of Cloud Virtual Machine”. In: *IEEE Access* 9 (2021), pp. 131476–131495. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3113714. URL: <https://ieeexplore.ieee.org/document/9540844/> (visited on 12/11/2025).

- [9] Shivan Singh et al. “ILP Optimized LSTM-based Autoscaling and Scheduling of Containers in Edge-cloud Environment”. In: *JTIT* 2 (June 2025), pp. 56–68. ISSN: 1899-8852, 1509-4553. DOI: 10.26636/jtit.2025.2.2088. URL: <https://www.jtit.pl/jtit/article/view/2088> (visited on 12/11/2025).
- [10] Zhiqiang Zhou et al. *AHPA: Adaptive Horizontal Pod Autoscaling Systems on Alibaba Cloud Container Service for Kubernetes*. arXiv:2303.03640 [cs]. Mar. 2023. DOI: 10.48550/arXiv.2303.03640. URL: <http://arxiv.org/abs/2303.03640> (visited on 12/11/2025).
- [11] Sujun Ghimire et al. “Deep learning CNN-LSTM-MLP hybrid fusion model for feature optimizations and daily solar radiation prediction”. en. In: *Measurement* 202 (Oct. 2022), p. 111759. ISSN: 02632241. DOI: 10.1016/j.measurement.2022.111759. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0263224122009629> (visited on 12/11/2025).
- [12] David Rolnick et al. *Experience Replay for Continual Learning*. arXiv:1811.11682 [cs]. Nov. 2019. DOI: 10.48550/arXiv.1811.11682. URL: <http://arxiv.org/abs/1811.11682> (visited on 12/11/2025).
- [13] Saba Aslam et al. “CEL: A Continual Learning Model for Disease Outbreak Prediction by Leveraging Domain Adaptation via Elastic Weight Consolidation”. en. In: *Interdiscip Sci Comput Life Sci* 17.2 (June 2025), pp. 390–408. ISSN: 1913-2751, 1867-1462. DOI: 10.1007/s12539-024-00675-2. URL: <https://link.springer.com/10.1007/s12539-024-00675-2> (visited on 12/11/2025).
- [14] Hongyan Hao et al. “Continual Learning in Predictive Autoscaling”. In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. arXiv:2307.15941 [cs]. Oct. 2023, pp. 4616–4622. DOI: 10.1145/3583780.3615463. URL: <http://arxiv.org/abs/2307.15941> (visited on 12/11/2025).
- [15] Kubernetes Documentation. *Horizontal Pod Autoscaling*. Accessed December 05, 2025. 2026. URL: <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/> (visited on 12/05/2025).
- [16] Alibaba Group. *cluster-trace-gpu-v2025 Dataset*. Accessed January 15, 2026. 2025. URL: <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2025> (visited on 01/15/2026).